■ Virgo SLM V1.0: Comprehensive Technical Report

Author: Punit Kumar Kashyap

Affiliation: Department of Engineering Physics, Indian Institute of Technology Dharwad (IIT Dharwad)

Date: July 2026

License: MIT License

Repository: https://github.com/punitxdev/VIRGO-SLM

Kaggle Model Hub: https://www.kaggle.com/models/punitkashyap2007/virgo-instruct

---

■ Executive Summary

Virgo SLM V1.0 is an open-source, 120 Million Parameter (120M) Small Language Model engineered completely from scratch using pure PyTorch. Developed by a first-year undergraduate student in Engineering Physics at IIT Dharwad, Virgo was conceived as a ground-up exploration into modern decoder-only Transformer architectures, Byte-Pair Encoding (BPE) tokenization, KV-cached inference engines, multi-stage alignment, and web application deployment.

Unlike black-box commercial endpoints, Virgo offers **100% architectural and implementation transparency**. Every mathematical component—from Rotary Position Embeddings (RoPE) and Root Mean Square Normalization (RMSNorm) to SwiGLU activations and Key-Value (KV) cache generation—is written explicitly without external abstraction frameworks (like Hugging Face `transformers` modeling wrappers).

■ Transparency Notice & Performance Framing

> *[!IMPORTANT]*
>
> ***Honest Framing & Reality Check***
>
> - ***Scale**: Virgo is a 120M parameter model (~120,458,752 non-embedding parameters). It is designed to run efficiently on standard CPUs and lightweight single-GPU environments.*
>
> - ***Capabilities vs. Boundaries**: Virgo is **not** a competitor to frontier multi-billion parameter models (e.g. Llama-3 70B, GPT-4, or Claude 3.5). While it displays strong capability in generating clean Python functions, SQL queries, and basic scientific definitions, it exhibits known limitations in multi-step mathematical calculations, long-context coherence beyond 1024 tokens, and potential repetitive loops if output generation bounds are severely constrained (< 64 max tokens).*
>
> - ***Primary Value**: The core objective of this project is educational rigor, technical transparency, and reproducible engineering proof-of-concept for small language modeling.*

■ Official Kaggle Checkpoints

All official model weights for the Virgo family are hosted on **Kaggle Models**:

| Model Variant | Internal Checkpoint | Kaggle Model Hub Link | Description |

| :--- | :--- | :--- | :--- |

| ■ **Virgo Instruct** | `virgo_instruct_tuning.pt` / `virgo_instruction_v2.pt` | kaggle.com/.../virgo-instruct | Instruction-following model fine-tuned for prompt execution |

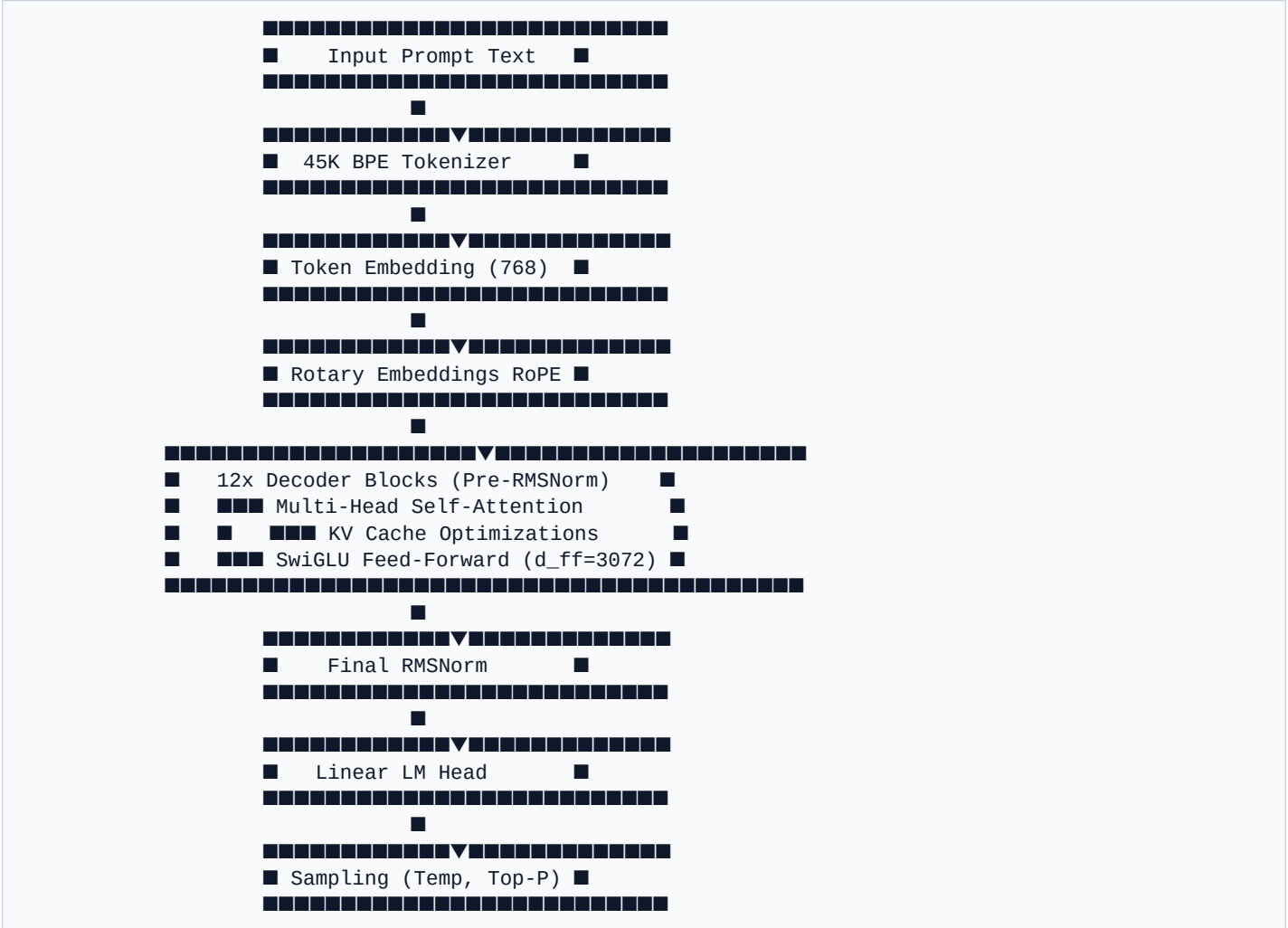
| ■ **Virgo Chat** | `virgo_chat_best.pt` | kaggle.com/.../virgo-chat | Multi-turn dialogue conversational alignment checkpoint |

| ■ **Virgo Align V1.0** | `virgo_IFT_ep2.pt` | kaggle.com/.../virgo-align-v1-0 | Preference and fine-grained alignment release |

| ■ **Virgo Base V1.0** | `virgo_IFT_ep1.pt` | kaggle.com/.../virgo-base-v1-0 | Raw pre-trained base model checkpoint (120M) |

■ Architectural Deep Dive

Virgo follows the modern autoregressive decoder-only Transformer paradigm inspired by Llama-style architectural enhancements.



1. Architectural Specifications Table

| Parameter | Specification | Description |

| :--- | :--- | :--- |

| **Architecture Type** | Decoder-only Transformer | Autoregressive language modeling |

| **Non-Embedding Parameters** | ~120,458,752 | Lightweight 120M class |

| **Transformer Layers (`n_layer`)** | 12 | Stacked decoder blocks |

| **Attention Heads (`n_head`)** | 12 | Multi-head self-attention |

| **Embedding Dimension (`d_model`)**| 768 | Model hidden representation width |

| **Head Dimension (`d_head`)** | 64 | $768 / 12 = 64$ per attention head |

| **Feed-Forward Dim (`d_ff`)** | 3,072 | SwiGLU intermediate dimension ($4 \times d_{model}$) |

| **Vocabulary Size** | 45,000 | Custom Byte-Level BPE |

| **Max Sequence Length** | 1,024 Tokens | Context window capacity |

| **Position Embeddings** | Rotary Position Embeddings (RoPE) | Relative positional encoding |

| **Layer Normalization** | RMSNorm | Pre-layer normalization with $\epsilon = 10^{-5}$ |

■ Mathematical Formulations

1. Root Mean Square Normalization (RMSNorm)

Standard LayerNorm centers feature activations by subtracting the mean and dividing by standard deviation. RMSNorm simplifies this by scaling inputs using only root-mean-square statistic, saving computational overhead without loss of model stability:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma$$

$$\text{where } \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^{d} x_i^2 + \epsilon}$$

- **Input vector x**: Feature activations of dimension $d = 768$.
- **Learnable parameter γ**: Element-wise scaling weight initialized to ones.
- **ϵ**: Numerical stability constant (10^{-5}).

2. SwiGLU Activation Function

Traditional Transformers use ReLU or GELU activations in Feed-Forward Networks (FFN). Virgo adopts **SwiGLU** (Swish Gated Linear Unit), which uses a dual-matrix projection gated by Swish:

$$\text{FFN}_{\text{SwiGLU}}(x) = \left(\text{Swish}(x W_g) \odot x W_v \right) W_o$$

$$\text{where } \text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

- $W_g \in \mathbb{R}^{768 \times 3072}$: Gate projection matrix.
- $W_v \in \mathbb{R}^{768 \times 3072}$: Value projection matrix.
- $W_o \in \mathbb{R}^{3072 \times 768}$: Output projection matrix.

3. Rotary Position Embeddings (RoPE)

Instead of adding absolute positional embeddings to input vectors, RoPE applies a rotation matrix to Query (Q) and Key (K) representations at each attention head:

$$R_{\Theta, m}^d = \begin{pmatrix}

\cos m\theta_1 & -\sin m\theta_1 & 0 & 0 \\

\sin m\theta_1 & \cos m\theta_1 & 0 & 0 \\

0 & 0 & \cos m\theta_2 & -\sin m\theta_2 \\

0 & 0 & \sin m\theta_2 & \cos m\theta_2

\end{pmatrix}$$

This formulation guarantees that the inner product between Q_m and K_n depends purely on relative distance $(m - n)$:

$$\langle R_{\Theta, m}^d Q, R_{\Theta, n}^d K \rangle = g(Q, K, m - n)$$

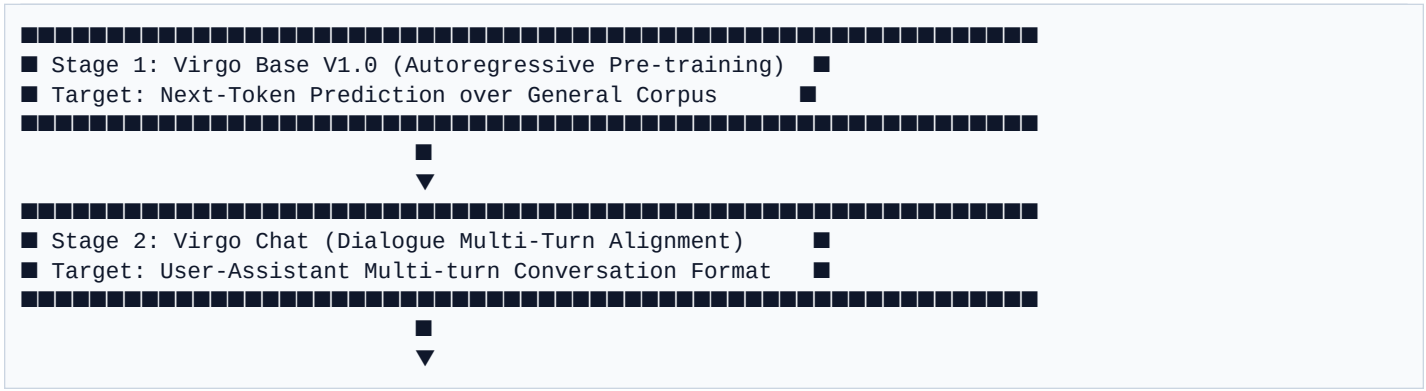
■ Tokenization Architecture

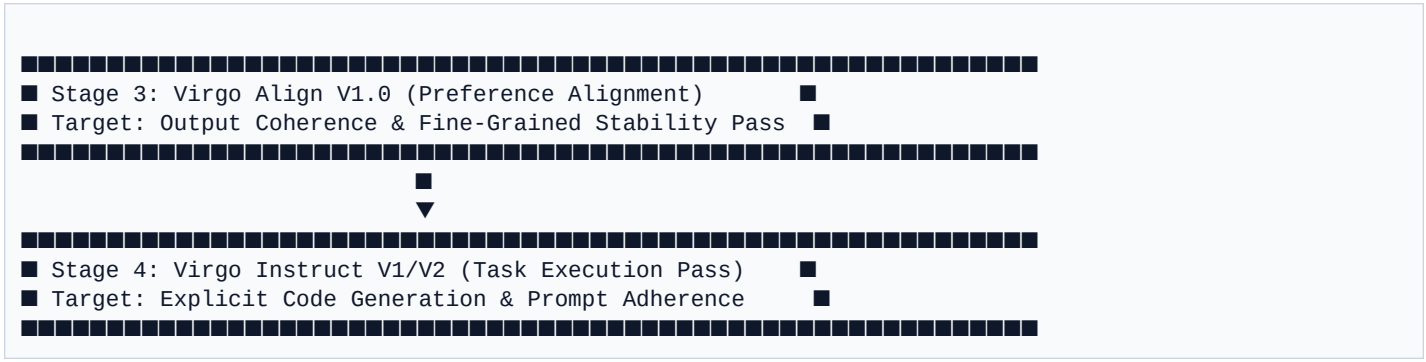
Virgo utilizes a custom **45,000 Byte-Level Byte-Pair Encoding (BPE)** vocabulary built with Hugging Face `tokenizers`:

- **Files**: `model/tokenizer.json`, `tokenizer/virgo_tokenizer.json`
- **Configuration**:
- `vocab_size`: 45,000
- `min_frequency`: 2
- `special_tokens`:
- (ID: 0)
- / (ID: 1)
- / (ID: 2)
- (ID: 3)
- **Tokenization Strengths**: High compression efficiency for Python code, mathematical formulas, Markdown formatting, and general English dialogue.

> *[!NOTE]*
>
> ***Data Privacy Notice**: The raw training datasets, instruction corpora, and pre-training text dumps used to train Virgo SLM V1.0 remain private and unreleased. Only the compiled model weights and tokenizer configurations are public.*

■ Multi-Stage Training Pipeline





1. **Pre-training (Virgo Base)**: Trained from scratch using Cross-Entropy Loss over autoregressive token sequences.

2. **Chat Fine-Tuning (Virgo Chat)**: Trained on multi-turn dialogue patterns with special role tokens (,).

3. **Alignment (Virgo Align)**: Refined for output formatting stability and token distribution smoothing.

4. **Instruction Post-Training (Virgo Instruct V1 & V2)**: Fine-tuned on task execution prompts (`virgo_instruct_tuning.pt` and `virgo_instruction_v2.pt`).

■ Inference Engine & Optimizations

The inference engine in `webapp/inference.py` is engineered for ultra-fast local execution:

1. **Key-Value (KV) Caching**:
   - Stores previously computed Key and Value tensors for tokens in the context window.
   - Reduces inference time complexity per generated token from $O(N^2)$ to $O(N)$.
2. **Sampling Controls**:
   - **Temperature Scaling**: Controls probability distribution sharpens/softens before softmax.
   - **Top-P (Nucleus) Sampling** (Default `0.9`): Cumulative probability thresholding.
   - **Top-K Sampling**: Truncates sampling pool to top-K candidates.
   - **Repetition Penalty** (Default `1.1`): Discount factor applied to previously emitted logits.
3. **Context Ceiling Management**:
   - Automatically truncates chat history iteratively if prompt length + `max_new_tokens` exceeds the 1,024 token window limit.

■ Interactive Web Studio Architecture

The project features a full-stack, local-first web application:

- **Backend**: FastAPI (`webapp/main.py`) with async endpoint routing, dynamic `MODEL_REGISTRY` model loading, and server middleware.
- **Frontend**: Glassmorphic UI (`webapp/static/index.html`, `style.css`, `script.js`) featuring ambient glow animations, category prompt suggestion cards, model switching selector, and responsive controls (Max Tokens, Temperature).

■ Comprehensive Evaluation & Known Behaviors

1. Benchmark Strengths

- **Python Function Generation**: Generates correct implementations for binary search, bubble sort, linked list operations, and string manipulation.
- **Scientific Concepts**: Precise concise descriptions for fundamental physics and chemistry queries.
- **Latency & Footprint**: Ultra-low memory usage (~1.4 GB model footprint) enabling instant CPU execution.

2. Known Limitations & Failure Modes

- **Complex Multi-Step Math**: Multiplication of numbers > 3 digits or multi-step algebraic operations can result in precision loss.
- **Tight Token Constraints**: Under generation bounds < 64 max tokens, the model may fail to output () early enough, causing token repetition loops.
- **Context Ceiling**: Maximum context is bounded to 1,024 tokens.

■ Future Expansion Roadmap

- [x] **Virgo Base V1.0 (120M)** — 12-layer decoder-only transformer pre-trained from scratch.
- [x] **Virgo Chat** — Multi-turn dialogue conversational alignment.
- [x] **Virgo Align V1.0** — Preference alignment pass.
- [x] **Virgo Instruct (V1 & V2)** — Task execution and instruction fine-tuning.
- [ ] **Virgo 350M Architecture** — Scaled 24-layer parameter expansion.
- [ ] **Virgo 1B Model** — 1 Billion parameter target architecture.
- [ ] **Virgo Multimodal (VLM)** — Vision-language embedding alignment.

■ Citation & Credits

If you use Virgo SLM V1.0 in your research, academic projects, or benchmark comparisons, please cite:

```
@misc{virgo2026report,
  author = {Punit Kumar Kashyap},
  title = {Virgo SLM V1.0: A 120M Parameter Small Language Model Built From Scratch},
  year = {2026},
  publisher = {GitHub},
  journal = {Technical Report},
  howpublished = {\url{https://github.com/punitxdev/VIRGO-SLM}},
  institution = {Indian Institute of Technology Dharwad}
}
```

Report compiled and published as part of the Virgo SLM V1.0 Open-Source Release.